EGC 211P C++ Mid-sem Cheatsheet
Kushman Goyal

1. UML → C++

- / red □	`private:` (class default)
# / yellow ◇	`protected:` (derived can use)
+ / green ○	`public:`
underlined	`static` + define outside once
italic method	`virtual`
italic = 0 / {abstract}	pure virtual ⇒ abstract
italic name / (A)	abstract class (can't `new` it)
`x: T`	`T x;`
`f(p: T) const: R`	`R f(T p) const;`
≪friend≫	`friend` declaration
≪std≫ box	library class: inherit, don't define

hollow triangle	`class D : public B`
filled diamond	owns: member object / `vector<T>`
hollow diamond	shares: `T*` / `shared_ptr<T>`
arrow A→B	pointer/reference member to B
dashed arrow	used only inside a method
1 / *	single member / `vector`

- Classes in the **question's order**. Write **every** listed member.
- Names, parameter order, `const`: copy exactly (tests call them).
- A name in the pseudocode but not in the UML = a **local variable**.

2. Class syntax

```
class Shape {                           // UML box
    static int count;                   // underlined -> static
    static const double PI;             // const static (non-int): define
   outside
    string name;                        // "- name: string" (private
   default)
protected:                              // "#"
    double scale;
public:                                 // "+"
    Shape() : name("none"), scale(1) { count++; }
    Shape(const string& n, double s = 1.0)
        : name(n), scale(s) { count++; }   // init list, same order as
   members
    virtual ~Shape() {}                 // any virtual -> virtual dtor
    string getName() const { return name; }  // getters: const
    static int getCount() { return count; }  // static: no 'this'
    virtual double area() const = 0;    // pure virtual -> abstract class
    virtual void print() const { cout << name << "\n"; }
};
int Shape::count = 0;                   // statics: outside, exactly once
const double Shape::PI = 3.14159;

class Circle : public Shape {           // ALWAYS public
    double r;
public:
    Circle(double r) : Shape("circle"), r(r) {} // base ctor first
    double area() const override { return 3.14159 * r * r * scale; }
    void print() const override { cout << "C "; Shape::print(); }
};
```

- A ctor you write removes the free default ctor. Need `A a;`? Add `A() {}`.
- `= default` leaves `int`/`double`/enum **garbage**: initialise every member in every ctor, or default it: `int count = 0;`
- Private members ARE inherited but not accessible: use getters, `#`, or the base ctor.
- A `const` method can't change members or call non-const methods. Mutators: no `const`.
- Virtual call inside a ctor is NOT polymorphic.

3. Exception classes

```
class AppException : public runtime_error {   // the standard pattern
public:
    AppException(const string& msg) : runtime_error(msg) {}
};
class NotFound : public AppException {        // hierarchy
public:
    NotFound(const string& msg) : AppException(msg) {}
};
class LimitEx : public AppException {         // with extra data
    double limit;
public:
    LimitEx(const string& m, double l) : AppException(m), limit(l) {}
    double getLimit() const { return limit; }
};
class RawEx : public exception {              // from std::exception
    string msg;
public:
    RawEx(const string& m) : msg(m) {}
    const char* what() const noexcept override { return msg.c_str(); }
};
```

Throw / catch

```
void demo() {
    try {
        throw NotFound("No such user");       // throw an OBJECT
    }
    catch (NotFound& e)     { cout << e.what(); }  // most-derived FIRST
    catch (AppException& e) { cout << e.what(); }
    catch (int code)        { cout << code; }      // throw paperID;
    catch (exception& e)    { cout << e.what(); }  // any std exception
    catch (...)             { throw; }             // anything; rethrow
}
```

- `: public` or `catch(runtime_error&)` misses it. Pass msg in init list (`runtime_error` has no default ctor).
- Catch by **reference**; order most-derived → base.
- `throw x;` matches only its exact type: `throw 5` → `catch(int)` only; `throw "s"` is `const char*`, not `string`.
- Code after `throw` never runs (no `return` needed).
- Library (`<stdexcept>`, don't define): `runtime_error`, `invalid_argument`, `out_of_range`, `logic_error`, `domain_error`, `length_error`, `overflow_error`. All have `(string)` ctor + `what()`.

Catch-and-log pattern (spec says "Handle Exception")

```
void addMoney(double amount, Logger& logger) {
    try {
        if (amount > 20000) throw AppException("Limit exceeded.");
        balance += amount;              // may itself throw
        logger.log(id, "Added Rs." + to_string(amount));
    } catch (AppException& e) {
        logger.log(id, string("Add-money failed: ") + e.what());
    }                                   // nothing escapes
}
```

4. Logger

```
class Logger {
    ofstream file;
    int count = 0;
public:
    Logger(const string& filename) : file(filename, ios::app) { // append
        if (!file.is_open()) throw runtime_error("Cannot open log file");
    }
    void log(const string& msg) {
        file << msg << "\n";
        file.flush();                   // written NOW, not at exit
        count++;
    }
    void log(const string& actor, const string& msg) {    // "[actor]:
   msg"
        file << "[" << actor << "]: " << msg << "\n";
        file.flush();
    }
    int getCount() const { return count; }
};
// Holding one:  class UPISystem { Logger logger{"./upi_log.txt"}; ... };
//          or:  Exchange(const string& f) : logger(f) {}   (no default
   ctor!)
// Passing one:  void addMoney(double a, Logger& logger)     (by
   REFERENCE)
```

- `ios::app` = append (keep old lines); no flag = truncate.
- `#include <fstream>`. A class holding an `ofstream` can't be copied: pass by `&`.
- Copy-paste log formats from the PDF. Check the spaces.

5. Smart pointers

```
#include <memory>
shared_ptr<Order> o = make_shared<BuyOrder>("alice", 100, 10);
o->getId();  *o -= 4;           // use like a pointer
vector<shared_ptr<Order>> book;
book.push_back(o);              // copy = SAME object, count 2
o.use_count();                  // 2
if (o == nullptr) {}            // default-constructed = null
Order* raw = o.get();           // raw pointer (don't delete it)
o.reset();                      // let go (count -1)

unique_ptr<Table> t = make_unique<Table>(5);  // single owner
unique_ptr<Table> t2 = move(t);               // transfer; t now null
// unique_ptr can't be copied: no t3 = t2;

weak_ptr<Order> w = o;                        // observes, doesn't own
if (shared_ptr<Order> p = w.lock()) { /* still alive */ }
```

`shared_ptr`	many owners; freed when the last one goes
`unique_ptr`	one owner; move only
`weak_ptr`	observer; `.lock()` to use

- Why: abstract class can't be stored by value (slicing); raw `T*` leaves "who deletes?".
- Polymorphism works: `shared_ptr<Base> p = make_shared<Derived>(...);` `p->virt();` (base needs a virtual dtor).
- Never `delete` a smart pointer. Read-only parameter: `const shared_ptr<T>&`.
- Raw-pointer owner (GIE `HashJoin`): `new` in ctor, `delete` in dtor.

6. Operators and friends

```
class Money {
    double amt;
public:
    Money(double a = 0) : amt(a) {}
    Money& operator+=(double x) { amt += x; return *this; }
    Money& operator-=(double x) { amt -= x; return *this; }
    Money operator+(const Money& o) const { return Money(amt + o.amt); }
    bool operator==(const Money& o) const { return amt == o.amt; }
    bool operator<(const Money& o) const { return amt < o.amt; }
    Money& operator++() { amt += 1; return *this; }        // ++m
    Money operator++(int) { Money old = *this; amt += 1; return old; } //
    m++
    friend ostream& operator<<(ostream& os, const Money& m) {
        os << "Rs." << m.amt;          // no 'this': use m.amt
        return os;                     // enables cout << a << b
    }
    friend class Bank;                 // every Bank method sees amt
};
// to a string (for logs):  ostringstream os; os << money; string line =
   os.str();
```

- `friend` line goes in the class **granting** access. One-way, not inherited.

- `operator<<` must be a free (friend) function: the left operand is `cout`, not your object.
- Compound ops (`+=`) return `*this` by **reference**; `+` returns a new value.
- Postfix `++` has the dummy `(int)` parameter.

7. Error: "no operator<< matches these operands"

Means: `cout` doesn't know how to print that type (an `enum class`, a vector, your class without `operator<<`, a `shared_ptr`'s object...).

```
enum class Side { BUY, SELL };          // values 0, 1
cout << s;                              // ERROR: no operator<< for Side
// Fix 1: switch
string toString(Side s) {
    switch (s) {
        case Side::BUY:  return "BUY";
        case Side::SELL: return "SELL";
    }
    return "?";
}
// Fix 2: lookup table (matches enum order)
const vector<string> sideName = {"BUY", "SELL"};
sideName[static_cast<int>(s)];
// Fix 3: if/else
string t; if (s == Side::BUY) t = "BUY"; else t = "SELL";
// Fix 4: your own operator (free function)
ostream& operator<<(ostream& os, Side s) { return os << toString(s); }
```

Prevent: print only strings and numbers; convert anything else first. Building a `string`? It must start as a `string`: `"#" + to_string(id) + ...`

8. STL you'll need

```
// erase by index (index loop, never inside a range-for)
for (int i = 0; i < (int)v.size(); i++)
    if (v[i]->getId() == id) { v.erase(v.begin() + i); break; }
// erase several matches
for (int i = 0; i < (int)v.size(); ) {
    if (bad(v[i])) v.erase(v.begin() + i); else i++;
}
// reverse loop: SIGNED int, never size_t
for (int i = (int)v.size() - 1; i >= 0; i--) v[i]->display();
// reference to one of two vectors, no ternary
vector<shared_ptr<Order>>* opp;                 // pointer: assign later
if (isBuy) opp = &asks; else opp = &bids;
for (auto& r : *opp) {}  (*opp)[i];  opp->erase(opp->begin() + i);

map<string, double> cash;
cash["alice"] = 100;            // [] CREATES missing key (needs default
   ctor)
if (cash.count("bob") == 0) {}  // existence check, no insert
cash.at("alice");               // throws out_of_range if missing
for (auto& kv : cash) cout << kv.first << " " << kv.second;  // sorted
   keys
map<string, OrderBook> books;   // OrderBook has no default ctor:
books.emplace(sym, OrderBook(sym));  books.at(sym).submit(o);
map<string, map<string, int>> pos;  pos[t][sym] += q;   // nested
map<double, pair<int,int>> lv;  lv[p].first += q;  lv[p].second++;
for (auto it = lv.rbegin(); it != lv.rend(); ++it)      // high -> low
    cout << it->first << " " << it->second.first << "\n";
```

Strings and numbers

```
string a = "Rs." + to_string(200.0);      // "Rs.200.000000" (6 dp!)
string b = string("Failed: ") + e.what(); // "lit" + char* won't compile
ostringstream os; os << fixed << setprecision(2) << 7.5;   // "7.50"
cout << fixed << setprecision(2) << x;   // STICKY for later doubles
double avg = sum / n;       // int/int = integer division! make sum double
```

9. My mistakes from the mocks

`paperID = uniq+1`	`++uniq` (counter never moved)
`nextId = 0` vs spec "from 1"	read the start value
`status` not set in ctor	init every member
`else throw` inside the find loop	throw **after** the loop
`!= A || != B`	`&&` (`||` is always true)
`== "Under review"` inverted	re-read: throw when NOT
compare pointers for duplicates	compare `->getName()`
`size_t i >= 0` reverse loop	`int i`
swapped print bodies	re-read each method name
`sum/size()` int division	`double sum`; check empty
`const` on a mutator	only for getters/display
`addRevewScore` typo	copy names from UML
retyped log strings	copy-paste
`showBalance` no `return`	compile with `-Wall`
logout via `Account*`	`Account*&` to change caller's ptr
range-for over a pointer	`*ptr`
erase inside range-for	index loop
uninit `T* p;`	`= nullptr`
half-typed line at the end	comment it out ⇒ compiles

10. Exam protocol

1. 15 min: read the WHOLE PDF. Mark check-order tables and output formats.
2. All class shells from UML, empty bodies (`return {};`) ⇒ **compile** (UML marks banked).
3. One method at a time, in question order; compile after each.
4. Run the local tester after each class.
5. Stuck >5 min: leave a compiling stub, move on (relative grading).
6. `g++ -std=c++17 -Wall -Wextra skeleton.cpp`
7. T−10 min: STOP. Comment out anything broken. Must compile alone. No `bits/stdc++.h`.